

MYSQL

- To open mysql in **windows** first we have to open xampp server and then click on mysql's start button.
- Then open command prompt and type :
mysql -u root
press enter
- To open mysql in Ubuntu open Terminal and type the following command :
sudo mysql
and press Enter

QUERIES :

DATABASE :-

- 1) create database abc;
- 2) drop database abc;
- 3) show databases;
- 4) use database abc;
- 5) show tables;
- 6) describe table_name;

TABLES :-

DDL Commands (data definition language, used to define and modify the structure of table)

1. CREATE :
create table demo(id INT PRIMARY KEY, name VARCHAR (25));
2. ALTER :
alter table demo add sub varchar(35);
alter table demo modify sub varchar(50);
alter table demo drop sub;
alter table demo rename to xyz;
3. TRUNCATE : (deletes all records at once but keeps table structure)
truncate table xyz;
4. RENAME :
rename table xyz to demo;
5. DROP : (deletes the entire table permanently with structure)
drop table demo;

DML Commands (data manipulation language, used to manipulate data stored in database tables)

1. INSERT :
insert into demo(id, name) values(1, 'Rida Patel');

2. UPDATE :
update demo set name = 'Sahil Khan' where id = 1;
3. DELETE :
delete from demo; //deletes all records
delete from demo where id = 1;
4. SELECT :
select * from demo;
select id, name from demo;

DCL Commands (data control language, used to control access and permissions on database objects)

(Privileges are : select, insert, update, delete)

1. GRANT :
grant select, insert ,update, delete on demo to 'user1';
2. REVOKE :
revoke delete, update on demo from 'user1';

TCL Commands (transaction control language, used to manage transactions in a database)

1. COMMIT : (saves all changes permanently in the database)
commit;
2. ROLLBACK : (undo changes made since the last commit)
rollback;
3. SAVEPOINT : (creates a point inside a transaction to roll back partially)
savepoint sp;
4. ROLLBACK TO SAVEPOINT : (undo changes upto a specific savepoint)
rollback to sp;
5. AUTO_COMMIT : (auto-commit is a mode in which every SQL statement is automatically saved in the database without using commit)

```
select @@autocommit; //check status (on or off)
set autocommit = 1;   //turn on
set autocommit = 0;   //turn off
```

SEQUENCE

A sequence is a user created database objects used to generate automatic unique numbers.

A sequence gives auto increment numbers, one after another.

Syntax : create sequence <sequence name>
start with 100
increment by 1
maxvalue 200

eg. suppose we have created seq named
sequence.

we can modify a created sequence by using

nocache;

alter sequence <seq> and remaining syntax..

- NEXTVAL : it is used to generate and return the next value of a sequence.
Insert into demo (id, name) values (seq.NEXTVAL , 'Rida')
- CURRVAL : it is used to return the current value of a sequence in the same session.
Select seq.CURRVAL from demo;

mysql does not supports sequence so we use auto-increment command

```
create table demo(id INT auto_increment PRIMARY KEY, name VARCHAR (25));  
insert into demo(name) values('Rida Patel');
```

VIEW

A view is defined as a database object that allows us to create a virtual table using select query, which does not store data physically but shows data from one or more tables.

A view is like a saved query that looks like a table. View is defined to hide complexity of query from the user.

```
create view v1 as select name from demo;  
create view v1 as select name from demo where marks >70;
```

```
select * from v1; //displaying  
drop view v1; //deleting
```

INDEX

An index is a database object. It can be created in table to access data in database more quickly and efficiently. Index is used to speed up the searching of records in a table.

1. Simple index : created on one column of a table and alllows duplicate values in indexed table

```
create index idx on demo(name);
```

2. Unique Index : does not allow duplicate values in the indexed column

```
create unique index idx on demo(id);
```

3. Composite index : created on multiple columns of a table

```
create index idx on demo(id, name);
```

```
show indexes from demo;  
drop index idx on table demo;            or            drop index idx;
```

JOINS

used to combine rows from two or more tables based on a related column. Join connects tables to get data from them together.

1. INNER JOIN
select s.Stud_id, s.Stud_name, s.Instr_id, i.Instr_name from student s inner Join instructor i ON s.Instr_id = i.Instr_id;
2. LEFT OUTER JOIN :
select s.Stud_id, s.Stud_name, s.Instr_id, i.Instr_name from student s Left Join instructor i ON s.Instr_id = i.Instr_id;
3. RIGHT OUTER JOIN :
select s.Stud_id, s.Stud_name, s.Instr_id, i.Instr_name from student s Right Join instructor i ON s.Instr_id = i.Instr_id;
4. FULL OUTER JOIN : (mysql does not support full outer join so we use union command to join right and left outer joins)
select s.Stud_id, s.Stud_name, s.Instr_id, i.Instr_name from student s Left Join instructor i ON s.Instr_id = i.Instr_id UNION select s.Stud_id,s.Stud_name, s.Instr_id, i.Instr_name from student s Right Join instructor i on s.Instr_id = i.Instr_id;

GROUP BY :

group by is used to group rows that have the same values in a column and apply aggregate functions on each group.

Note: used with aggregate functions.

```
select Country, count(*) from emp group by Country;
```

HAVING :

having is used to filter groups, not individual rows.

```
select Country, count(*) from emp group by Country having count(*)<5;
```

ORDER BY :

order by is used to sort the result set in ascending or descending order.

```
select EmpName,salary from emp order by salary asc;  
select EmpName,salary from emp order by salary desc;
```

imp :

where : filters rows

having : filters groups

group by comes before having

order by comes at the end

AGGREGATE FUNCTIONS :

1. COUNT() :
select count(salary) from emp;
2. SUM() :
select sum(salary) from emp;
3. AVG() :
select avg(salary) from emp;
4. MIN() :
select min(salary) from emp;
5. MAX() :
select max(salary) from emp;

SET OPERATIONS

set operations in sql are used to combine the results of two or more select queries into a single result set

rules :

- each select must have same no. of columns
- corresponding columns must have same data type
- column names are taken from the first select
- order by is used only at the end

Tyes :-

1. UNION : union is used in sql to combine the results of two select queries and show the common values only by removing duplicate entries
select * from table1 UNION select * from table2;
2. INTERSECT : intersect is a set operation in sql that returns only the common records present in both select queries.
select * from table1 INTERSECT select * from table2;
3. EXCEPT : except is a set operation in sql that returns the records from the first select query that are not present in the second select query
select * from table1 EXCEPT select * from table2;

ORACLE (PL/SQL)

Assignment no. 05 : PL?SQL code for use of control structure and exception handling is mandatory

Borrower and Fine

code:

```
CREATE TABLE BORROWER(  
  ROLLNO NUMBER,  
  NAME VARCHAR(30),  
  ISSUEDATE DATE,  
  BOOK VARCHAR(30),  
  STATUS VARCHAR(10)  
);
```

```
CREATE TABLE FINE(  
  ROLLNO NUMBER,  
  DATES DATE,  
  AMOUNT NUMBER  
);
```

```
DECLARE  
  V_ROLLNO NUMBER := :ROLLNO;  
  V_NAME VARCHAR(30) := :NAME;  
  V_ISSUEDATE DATE := TO_DATE(:ISSUEDATE, 'DD-MM-YYYY');  
  V_BOOK VARCHAR(30) := :BOOK;  
  V_STATUS VARCHAR(10) := :STATUS;
```

```
BEGIN  
  INSERT INTO BORROWER (ROLLNO, NAME, ISSUEDATE, BOOK, STATUS)  
  VALUES(V_ROLLNO, V_NAME, V_ISSUEDATE, V_BOOK, V_STATUS);  
  DBMS_OUTPUT.PUT_LINE('RECORD INSERTED SUCCESSFULLY');  
  
END;  
/
```

```
DECLARE  
  V_ROLLNO NUMBER := :ROLLNO;  
  V_ISSUEDATE DATE;  
  V_STATUS VARCHAR(10);  
  V_DAYS NUMBER;  
  V_AMOUNT NUMBER := 0;
```

```
BEGIN  
  SELECT ISSUEDATE, STATUS INTO V_ISSUEDATE, V_STATUS FROM BORROWER WHERE  
  ROLLNO = V_ROLLNO;
```

```
IF V_STATUS = 'ISSUED' THEN V_DAYS := TRUNC(SYSDATE - V_ISSUEDATE);
```

```

IF V_DAYS BETWEEN 15 AND 30 THEN V_AMOUNT := V_DAYS * 5;

ELSIF V_DAYS > 30 THEN V_AMOUNT := V_DAYS * 50;

END IF;

UPDATE BORROWER SET STATUS = 'RETURNED' WHERE ROLLNO = V_ROLLNO;

INSERT INTO FINE(ROLLNO, DATES, AMOUNT) VALUES(V_ROLLNO, SYSDATE,
V_AMOUNT);

DBMS_OUTPUT.PUT_LINE('FINE AMOUNT : '||V_AMOUNT);

END IF;

END;
/
SELECT * FROM BORROWER;
SELECT * FROM FINE;

```

Assignment no. 06 : Cursor

eg. code :

```

create table employees(
id number,
name varchar(30),
hdate date
);

declare
v_id number := :id;
v_name varchar(30) := :name;
v_hdate date := To_Date(:hdate, 'dd-mm-yyyy');

begin
insert into employees(id, name, hdate) values(v_id, v_name, v_hdate);

end;
/

declare
cursor emp is select id, name, hdate from employees;

v_id employees.id%TYPE;
v_name employees.name%TYPE;
v_hdate employees.hdate%TYPE;
v_month number;

```

```

v_incentive number;

begin
open emp;
loop
fetch emp into v_id, v_name, v_hdate;
exit when emp%NOTFOUND;

v_month := (sysdate - v_hdate)/30;

if v_month >= 10 then v_incentive := 10000;
elsif v_month >= 5 then v_incentive := 5000;
else v_incentive := 2000;
end if;

dbms_output.put_line('Emp Id : '||v_id);
dbms_output.put_line('Emp Name : '||v_name);
dbms_output.put_line('Hired Date : '||v_hdate);
dbms_output.put_line('Incentive Amount : '||v_incentive);

end loop;
close emp;
end;
/
select * from employees;

```

Assignment no. 07 : Trigger (Book and Library Audit)

Code :

```

create table book(
accno number,
title varchar(20),
author varchar(50),
publisher varchar(25),
count number
);

create table library_audit(
accno number,
title varchar(30),
author varchar(50),
publisher varchar(25),
count number,
action_date date,
status varchar(20)
);

```


create or replace trigger deletion
before delete on book
for each row

```
begin
insert into library_audit(accno, title, author, publisher, count, action_date, status)
values(
:old.accno,
:old.title,
:old.author,
:old.publisher,
:old.count,
SYSdate,
'Deleted'
);
dbms_output.put_line('Deletion Trigger Fired');
end;
/
```

create or replace trigger updation
before update on book
for each row

```
begin
insert into library_audit(accno, title, author, publisher, count, action_date, status)
values(
:new.accno,
:new.title,
:new.author,
:new.publisher,
:new.count,
sysdate,
'Updated'
);
dbms_output.put_line('Updation Trigger Fired');
end;
/
```

```
insert into book values(101, 'DBMS', 'Avhad', 'KKW', 25);
insert into book values(102, 'OOP', 'Deshmukh', 'KKW', 12);
insert into book values(103, 'DCN', 'Bairagi', 'KKW', 50);
insert into book values(104, 'DSA', 'Khaire', 'KKW', 69);
insert into book values(105, 'SE', 'Shaikh', 'KKW', 75);
```

```
select * from book;
```

```
update book set count = 25 where accno = 105;
delete from book where title = 'DSA';
select * from library_audit;
```

MONGODB

Assignment no. 09 (mongodb)

```
mongosh
use RIDA;
```

1. RIDA> db.CreateCollection(Book)
2. db.Book.insertMany([{Title:"DBMS",Description:"Database Management System",By:"Avhad Mam",URL:"kkw.edu.in",Tags:"sql",Likes:"50"},
 {}, {}, {}, {}])
3. db.Book.find()
4. db.Book.insertOne([{Title:"DCN",Description:"Data Communication & Networking",By:"BairagiMam",URL:"kkw.edu.in",Tags:"dcn",Likes:"29",Username:"ridafatem a29",Comments:"Excellent"}])
5. db.Book.find({Title:"DBMS"})
6. db.Book.find({\$or:[{By:"Kalyani Mam"},
7. {Title:"DS"}]}))
8. db.Book.find({\$and:[{By:"Kalyani Mam"}, {Title:"DS"}]}))
9. db.Book.find({Likes:{\$gt:"25"}})
10. db.Book.find({Likes:{\$lt:"30"}})
11. db.Book.find({Likes:{\$gte:"30"}})
12. db.Book.find({Likes:{\$lte:"30"}})
13. db.Book.updateMany({Title:"DS"},{\$set: {Title:"DSA"}});
14. db.Book.deleteMany({Title:"DSA"});
15. db.Book.find({By:"Avhad Mam"}).skip(1).limit(1);
16. db.Book.find().sort({Title :1});
17. db.Book.find().sort({Title :-1});

```
db.Book.drop()
```

Assignment no. 10 : Aggregation and Indexing

```
db.Book.aggregate({$match : {By : "Kalyani Mam"}},{ $count : "TotalBooks"})
```

```
db.Book.aggregate([{$match : {By : "Avhad Mam"}},{ $group : {_id : "$By", MinLikes :  
{$min : "$Likes"},MaxLikes : {$max : "$Likes"}}}])
```

```
db.Book.aggregate([{$match : {By : "Avhad Mam"}},{ $group : {_id : "$By", AvgLikes :  
{ $avg : "$Likes"}}}])
```

note : This is a string, not a number → \$avg cannot calculate → result = null

```
db.Book.aggregate([{$match : {By : "Avhad Mam"}},{ $group : {_id : "$By", Last : {$last :  
"$Likes"},First : {$first : "$Likes"}}}])
```

```
db.Book.aggregate([{$match : {By : "Kalyani Mam"}},{ $group : {_id : "$By", Last : {$last :  
"$Title"},First : {$first : "$Title"}}}])
```